



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 互斥与锁

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 2025/05/10

Section 1 实验概述

在本次实验中，我们首先使用硬件支持的原子指令来实现自旋锁 `SpinLock`，自旋锁将成为实现线程互斥的有力工具。接着，我们使用 `SpinLock` 来实现信号量，最后我们使用 `SpinLock` 和信号量来给出两个实现线程互斥的解决方案。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 虚拟机版本/处理器型号：Virtualbox
 - 代码编辑环境：Vscode+nasm+C/C++插件
 - 代码编译工具：gcc/g++（64 位）
 - 重要三方库信息：

Section 3 实验任务

该节描述需要完成的几个实验任务，即重述实验题目的总要求，建议使用项目编号分点阐述。详细要求可在下一节【实验步骤与实验结果】中列出。

- 实验任务 1：自旋锁与信号量的实现
- 实验任务 2：生产者-消费者问题
- 实验任务 3：哲学家就餐问题

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：

子任务 1：利用自旋锁和信号量实现同步

在实验 6 中，我们实现了自旋锁和信号量机制。现在，请同学们分别利用指导书中实现的自旋锁和信号量方法，解决实验 6 指导书中的“消失的芝士汉堡”问题，保存结果截图并说说你的总体思路。注意：请将你姓名的英文缩写包含在某个线程的输出信息中（比如代替母亲或者儿子），用作结果截图中的个人信息表征。

子任务 2-自实现锁机制

实验 6 教程中使用了原子指令 `xchg` 来实现自旋锁。但这种方法并不是唯一

的。例如，x86 指令中提供了另外一个原子指令 `bts` 和 `lock` 前缀等，这些指令也可以用来实现锁机制。现在，同学们需要结合自己所学的知识，实现一个与指导书实现方式不同的锁机制。最后，尝试用你实现的锁机制解决“消失的芝士汉堡”问题，保存结果截图并说说你的总体思路。

- 思路分析：

在“消失的芝士汉堡”问题中，核心是两个线程对共享变量 `cheese_burger` 的并发访问未做同步，导致 `Mother` 离开临界区后，`Boy` 可以趁机修改数据，从而产生竞态条件。为了解决这个问题，我们分别使用了自旋锁和信号量两种同步机制。自旋锁通过原子交换指令 `xchg` 实现互斥，线程在获取锁失败时会持续循环等待，适用于临界区很短的情况。信号量则通过维护一个资源计数器实现同步访问，当资源不可用时，线程会阻塞等待，不会占用 CPU 资源。这两种方法在解决该问题时均能保证 `Mother` 线程在完成操作前 `Boy` 无法干预，从而避免汉堡消失的问题。在实验过程中，我在 `Boy` 的输出中加入了自己的英文缩写以表明线程身份。

为了进一步加深对底层原子操作的理解，我们在第二个子任务中尝试使用 `bts` 指令来自行实现一个新的锁机制。与 `xchg` 类似，`bts` 是一种原子位操作指令，可用于测试并设置锁变量的特定位。我们利用 `bts [lock], 0` 来判断并设置锁状态，如果目标位原本是 0（未加锁），就原子性地将其置为 1，并认为加锁成功；否则表示锁已被占用，当前线程需等待。这种方式本质上仍为自旋锁，但实现更加底层、贴近硬件逻辑，且具备拓展性。我们复用了原有自旋锁类的结构，仅修改其 `lock` 函数中的汇编实现，并通过运行同样的汉堡问题验证同步效果与原方法一致，达到解决竞态的目的。

- 实验步骤：给出关键部分的代码实现/命令行的执行过程，辅以必要的文字描述（可从指导书上摘抄）。

1. “消失的芝士汉堡”案例

- 1.1 `Mother` 的线程

```
int cheese_burger;

void a_mother(void *arg)
{
    int delay = 0;
```

```

printf("mother: start to make cheese burger, there are %d cheese burger now\n", cheese_burger);

// make 10 cheese_burger
cheese_burger += 10;

printf("mother: oh, I have to hang clothes out.\n");

// hanging clothes out
delay = 0xffffffff;
while (delay)
    --delay;

// done
printf("mother: Oh, Jesus! There are %d cheese burgers\n", cheese_burger);
}

```

1.2 Boy 线程

```

void a_naughty_boy(void *arg)
{
    printf("boy   : Look what I found!\n");

    // eat all cheese_burgers out secretly
    cheese_burger -= 10;

    // run away as fast as possible
}

```

1.3 Thread

创建 Mother 线程和 Boy 线程

```

void first_thread(void *arg)
{
    // 第 1 个线程不可以返回
    stdio.moveCursor(0);

    for (int i = 0; i < 25 * 80; ++i)
    {
        stdio.print(' ');
    }

    stdio.moveCursor(0);

    cheese_burger = 0;

    programManager.executeThread(a_mother, nullptr, "second thread", 1);
    programManager.executeThread(a_naughty_boy, nullptr, "third thread", 1);

    asm_halt();
}

```

1.4 运行结果

Mother 洗衣服回来，出现了汉堡消失的问题。

```
QEMU
LinHongYumother: start to make cheese burger, there are 0 cheese burger now
LinHongYumother: oh, I have to hang clothes out.
LinHongYuboy    : Look what I found!
LinHongYumother: Oh, Jesus! There are 0 cheese burgers
```

2. 自旋锁解决“消失的芝士汉堡”

2.1 创建自旋锁的类

```
#ifndef SYNC_H
#define SYNC_H

#include "os_type.h"

class SpinLock
{
private:
    // 共享变量
    uint32 bolt;
public:
    SpinLock();
    void initialize();
    // 请求进入临界区
    void lock();
    // 离开临界区
    void unlock();
};

#endif
```

2.2 自旋锁的定义

初始化

```
SpinLock::SpinLock()
{
    initialize();
}

void SpinLock::initialize()
{
    bolt = 0;
}
```

进入临界区上锁

```
void SpinLock::lock()
{
    uint32 key = 1;
    do
    {
```

```

        asm_atomic_exchange(&key, &bolt);

    } while (key);
}

```

其中 `asm_atomic_exchange` 需要是一个原子操作

```

; void asm_atomic_exchange(uint32 *register, uint32 *memory);
asm_atomic_exchange:

    push ebp
    mov ebp, esp
    pushad

    mov ebx, [ebp + 4 * 2] ; register
    mov eax, [ebx]        ; 取出 register 指向的变量的值
    mov ebx, [ebp + 4 * 3] ; memory
    xchg [ebx], eax       ; 原子交换指令
    mov ebx, [ebp + 4 * 2] ; memory
    mov [ebx], eax        ; 将 memory 指向的值赋值给 register 指向的变量

    popad
    pop ebp
    ret

```

离开临界区释放

```

void SpinLock::unlock()
{
    bolt = 0;
}

```

2.3 使用自旋锁解决问题

Mother 线程

```

void a_mother(void *arg)
{
    aLock.lock();
    int delay = 0;

    printf("mother: start to make cheese burger, there are %d cheese burger now\n", cheese_burger);
    // make 10 cheese_burger
    cheese_burger += 10;
    printf("mother: oh, I have to hang clothes out.\n");
    // hanging clothes out
    delay = 0xffffffff;
    while (delay)
        --delay;
    // done
    printf("mother: Oh, Jesus! There are %d cheese burgers\n", cheese_burger);
    aLock.unlock();
}

```

```
}
```

Boy 线程

```
void a_naughty_boy(void *arg)
{
    aLock.lock();

    printf("boy   : Look what I found!\n");

    // eat all cheese_burgers out secretly

    cheese_burger -= 10;

    // run away as fast as possible

    aLock.unlock();
}
```

使用 Thread 创建上面两个线程

```
void first_thread(void *arg)
{
    // 第1个线程不可以返回

    stdio.moveCursor(0);

    for (int i = 0; i < 25 * 80; ++i)
    {
        stdio.print(' ');
    }

    stdio.moveCursor(0);

    cheese_burger = 0;

    aLock.initialize();

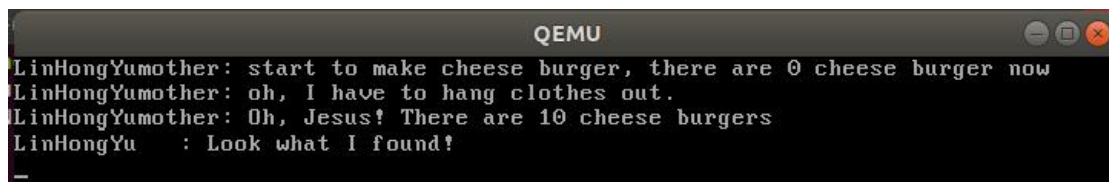
    programManager.executeThread(a_mother, nullptr, "second thread", 1);

    programManager.executeThread(a_naughty_boy, nullptr, "third thread", 1);

    asm_halt();
}
```

2.4 运行结果

可以观察结果，Mother 洗衣服的时候，由于汉堡上锁了，所以 Boy 无法偷吃，Mother 洗衣服结束时仍然可以观察到 10 个 burgers，后续 Boy 才可以吃汉堡。



```
QEMU
LinHongYumother: start to make cheese burger, there are 0 cheese burger now
LinHongYumother: oh, I have to hang clothes out.
LinHongYumother: Oh, Jesus! There are 10 cheese burgers
LinHongYu   : Look what I found!
```

3. 信号量解决“消失的芝士汉堡”

3.1 信号量的定义

```
class Semaphore
{
private:
    uint32 counter;

    List waiting;
```

```

    SpinLock semLock;

public:
    Semaphore();
    void initialize(uint32 counter);
    void P();
    void V();
};

```

3.2 信号量的实现

初始化

```

Semaphore::Semaphore()
{
    initialize(0);
}

void Semaphore::initialize(uint32 counter)
{
    this->counter = counter;
    semLock.initialize();
    waiting.initialize();
}

```

P 操作

```

void Semaphore::P()
{
    PCB *cur = nullptr;

    while (true)
    {
        semLock.lock();

        if (counter > 0)
        {
            --counter;
            semLock.unlock();
            return;
        }

        cur = programManager.running;
        waiting.push_back(&(cur->tagInGenerallist));
        cur->status = ProgramStatus::BLOCKED;
        semLock.unlock();
        programManager.schedule();
    }
}

```


V 操作

```
void Semaphore::V()
{
    semLock.lock();
    ++counter;
    if (waiting.size())
    {
        PCB *program = ListItem2PCB(waiting.front(), tagInGenerallist);
        waiting.pop_front();
        semLock.unlock();
        programManager.MESA_Wakeup(program);
    }
    else
    {
        semLock.unlock();
    }
}
```

其中线程的唤醒 WakeUp

```
void ProgramManager::MESA_Wakeup(PCB *program) {
    program->status = ProgramStatus::READY;
    readyPrograms.push_front(&(program->tagInGenerallist));
}
```

3.3 信号量解决“消失的芝士雪豹”问题

Mother 线程

```
void a_mother(void *arg)
{
    semaphore.P();
    int delay = 0;

    printf("mother: start to make cheese burger, there are %d cheese burger now\n", cheese_burger);
    // make 10 cheese_burger
    cheese_burger += 10;
    printf("mother: oh, I have to hang clothes out.\n");
    // hanging clothes out
    delay = 0xffffffff;
    while (delay)
        --delay;
    // done
    printf("mother: Oh, Jesus! There are %d cheese burgers\n", cheese_burger);
    semaphore.V();
}
```

Boy 线程

```

void a_naughty_boy(void *arg)
{
    semaphore.P();

    printf("boy   : Look what I found!\n");

    // eat all cheese_burgers out secretly
    cheese_burger -= 10;

    // run away as fast as possible
    semaphore.V();
}

```

Thread 创建上面两个线程

```

void first_thread(void *arg)
{
    // 第1个线程不可以返回
    stdio.moveCursor(0);

    for (int i = 0; i < 25 * 80; ++i)
    {
        stdio.print(' ');
    }

    stdio.moveCursor(0);

    cheese_burger = 0;
    semaphore.initialize(1);

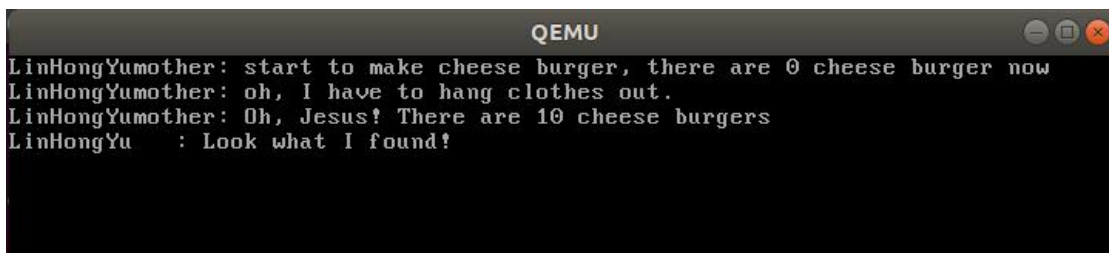
    programManager.executeThread(a_mother, nullptr, "second thread", 1);
    programManager.executeThread(a_naughty_boy, nullptr, "third thread", 1);

    asm_halt();
}

```

3.4 运行结果

效果同自旋锁效果一致，不再赘述



```

QEMU
LinHongYumother: start to make cheese burger, there are 0 cheese burger now
LinHongYumother: oh, I have to hang clothes out.
LinHongYumother: Oh, Jesus! There are 10 cheese burgers
LinHongYu      : Look what I found!

```

4. 自实现锁机制

4.1 “原子指令”问题

之前的锁并不是一个原子指令，要修改的值值发生更改之后，没有办法阻止其他进程也修改自身的值，因此不是一个原子指令，无法真正的实现互斥。

4.2 bts 指令

bts 可以对内存直接操作。

bts 检查一个位是不是为 1:

如果是 1, CF 位置 1, 原位不改变。

如果是 0, CF 位置 0, 原位被改变为 1。

4.3 自实现锁

4.3.1 mov ebx, [ebp + 12] (第一个参数):

取出锁变量地址, 通常这个地址指向一个 4 字节整型, 表示锁的状态 (如 0 表示未上锁, 1 表示已上锁)。

4.3.2 bts dword [ebx], 0 (Bit Test and Set):

测试并设置第 0 位, 它是原子操作 (硬件保证不被中断打断)。

如果该位原来是 1, 则设置 CF = 1, 说明锁已经被占用。

如果该位原来是 0, 就将它设置为 1, CF = 0, 表示我们成功获取锁。

4.3.3 jc asm_atomic_exchange_exit:

如果 CF 被设置 (表示原来是 1), 跳转退出, 不做任何处理 (加锁失败)。

否则继续执行, 说明锁获取成功。

```
; my_asm_atomic_exchange
asm_atomic_exchange:
    push ebp
    mov ebp, esp
    pushad
    mov ebx, [ebp + 4 * 3]
    bts dword [ebx], 0
    jc asm_atomic_exchange_exit
    mov ebx, [ebp + 4 * 2]
    mov dword [ebx], 0
asm_atomic_exchange_exit:
    popad
    pop ebp
    ret
```

4.4 应用解决“消失的芝士雪豹”问题

直接使用原来的自旋锁的框架进行应用, 只是修改了上面这一部分的代码。

4.5 运行结果

```
QEMU
LinHongYumother: start to make cheese burger, there are 0 cheese burger now
LinHongYumother: oh, I have to hang clothes out.
LinHongYumother: Oh, Jesus! There are 10 cheese burgers
LinHongYu      : Look what I found!
```

----- 实验任务 2 -----

● 任务要求：生产者与消费者问题

问题描述：某位商人在餐厅举行生日宴会。餐桌上有一个点心盘，最多可以容纳 5 块蛋糕，每个人（服务生或者来宾）每次只能放入/拿出 1 块蛋糕。服务生 A 负责向点心盘中放入抹茶蛋糕，服务生 B 负责向点心盘中放入芒果蛋糕。生日宴会有 6 位男性来宾，4 位女性来宾，男性来宾等待享用抹茶蛋糕，女性来宾等待享用芒果蛋糕。如果盘中没有对应口味的蛋糕且点心盘没有放满，来宾会给相应的服务生发送一个请求服务信号，服务生受到信号会放入 1 块蛋糕。

子任务 1：线程的竞争与冲突

在子任务 1 中，要求不使用任何实现同步/互斥的工具。因此，不同的线程之间可能会产生竞争/冲突，从而无法达到预期的运行结果。请同学们将线程竞争导致错误的场景呈现出来，保存相应的截图，并描述发生错误的场景。（提示：可通过输出共享变量的值进行观察）

子任务 2：利用信号量解决问题

针对你选择的问题场景，简单描述该问题中各个线程间的互斥关系，并使用信号量机制实现线程的同步。说说你的实现方法，并保存能够证明你成功实现线程同步的结果截图。

● 思路分析：

在本实验的第一个子任务中，我们需要模拟一个未加同步的生产者-消费者模型，以观察线程竞争造成的错误。在设定中，男性来宾只吃抹茶蛋糕，女性来宾只吃芒果蛋糕，两个服务生负责分别生产这两种蛋糕。当不使用任何同步手段时，多个线程可能同时判断点心盘未滿（即 $\text{matcha_count} + \text{mango_count} < \text{plate_capacity}$ ），并同时执行放蛋糕操作，造成蛋糕总数超过盘子容量。实验中通过加入延迟（delay）故意拉长临界区执行时间，使得竞争条件更容易发生，从而观察到点心盘中蛋糕数量超过 5 的非法情况。这种结果揭示了线程之间在访

间共享变量时缺乏互斥保护可能引发的不一致状态。

在第二个子任务中，我们引入信号量机制以实现线程同步，从而避免竞态条件的发生。具体地，使用一个初始值为 5 的信号量来控制对蛋糕盘的访问，每当一个线程要对蛋糕进行增减操作时，需先获取信号量（P()），操作完成后释放信号量（V()）。通过这种机制，确保了任一时刻只有一个线程能进入临界区操作共享变量，避免了多个线程同时判断并写入的问题。同时仍保留了原有的逻辑：如果对应口味的蛋糕为 0，来宾可请求服务生补充蛋糕。运行结果表明，加入信号量后，盘中蛋糕数量始终维持在 0 到 5 之间，系统状态保持一致，从而成功解决了多线程访问共享资源时的互斥问题。

- 实验步骤：

1. 子任务一 线程的竞争和冲突

- 1.1 代码

共享变量

```
int matcha_count;
int mango_count;
int plate_capacity;
```

男士和女士进程

```
void man(void *arg)
{
    if (matcha_count > 0)
    {
        matcha_count = matcha_count - 1;
        printf(" %d ", matcha_count);
    }
    else
    {
        if(matcha_count + mango_count < plate_capacity)
        {
            matcha_count = matcha_count + 1;
            printf(" %d ", matcha_count);
        }
    }
}

void woman(void *arg)
{
    if (mango_count > 0)
```

```

{
    mango_count = mango_count - 1;
    printf(" %d ", mango_count);
}
else
{
    if(matcha_count + mango_count < plate_capacity)
    {
        mango_count = mango_count + 1;
        printf(" %d ", mango_count);
    }
}
}
}

```

需要使用 delay 形成缓冲，不然进程执行太快，无法观察到进程存在竞争的情况

```

delay = 0xffffffff;
while (delay)
    --delay;

```

1.2 结果展示



输出的共享变量为桌子上的蛋糕数，不使用信号量解决问题时可能产生盘子蛋糕超过 5 个的情况，但是盘子只能装五个蛋糕。

其是因为两个服务生同时检测 `matcha_count + mango_count < plate_capacity` 为真，然后同时放入，导致总数超过 5。

2. 子任务二 信号量解决问题

2.1 代码

使用信号量

在进程添加 P 操作和 V 操作

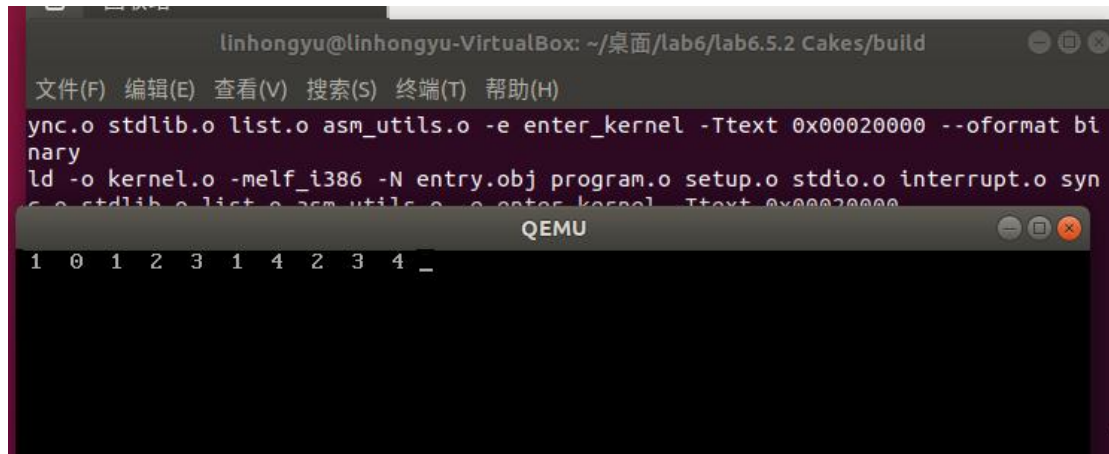
```

semaphore.P();

semaphore.V();

```

2.2 结果展示



可以观察到桌子上的蛋糕不会超出 5 个，不会产生原来的问题。

----- 实验任务 3 -----

● 任务要求：哲学家就餐问题

问题场景

假设有 5 位哲学家，他们在就餐时只能思考或者就餐。这些哲学家公用一个圆桌，每个哲学家都坐在一把指定的椅子上。在桌子上放着 5 根筷子，每两根筷子之间都放着一碗葱油面。下面是一些约束条件：

1. 当一位哲学家处于思考状态时，他对其他哲学家不会产生影响
2. 当一位哲学家感到饥饿时，他会试图拿起与他相邻的两根筷子
3. 一个哲学家一次只能拿起一根筷子，在拿到两根筷子之前不会放下手里的筷子
4. 如果筷子在其他哲学家手里，则需等待
5. 当一个饥饿的哲学家同时拥有两根筷子时，他会开始吃面
6. 吃完面后的哲学家会同时放下两根筷子，并开始思考

子任务 1：简单解决方法

同学们需要在实验 6 教程的代码环境下，创建多个线程来模拟哲学家就餐的场景。然后，同学们需要结合信号量来实现理论教材（参见《操作系统概念》中文第 9 版 187 页）中给出的关于哲学家就餐的简单解决办法。最后，保存结果截图并说说你是怎么做的。

注意：

1. 可以通过输出不同哲学家的状态信息，验证你使用教程的方法确实能解决哲学家就餐问题。

2. 请将你的学号的后四位包含在其中一个哲学家线程的输出信息中，用作结果截图的个人信息表征。

子任务 2：死锁应对策略（选做）

策略 1：利用抽屉原理（鸽笼原理）。即允许最多 4 个哲学家同时做在桌子上，保证至少有 1 位哲学家能够吃到面。

● 思路分析：

避免死锁的简单解决方法：

在最初的哲学家就餐模型中，每位哲学家都按照相同的顺序依次尝试获取左边和右边的筷子，这会导致一种典型的死锁情况：每位哲学家拿到了左手的筷子后都在等待右手的筷子，系统陷入互相等待的僵局。为了解决这个问题，我们采用了教材中提出的简单策略，即打破“环路等待”条件：只需让其中一位哲学家反向获取筷子的顺序（例如先拿右筷子再拿左筷子），就可以从逻辑上打破资源分配的闭环，从而彻底避免死锁。该方法简单有效，能够确保哲学家之间不会永远相互等待。

抽屉原理策略：

为了进一步保障系统不会进入死锁状态，我们采用了“抽屉原理”（鸽笼原理）策略：限制最多只有 4 位哲学家同时尝试进餐。因为同时有 5 把椅子和 5 根筷子，最多允许 4 位哲学家尝试就餐可以确保系统中始终有一位哲学家可以成功获得两根筷子。一旦这位哲学家吃完后释放筷子，其他人即可继续就餐，从而从系统资源调度的角度避免死锁。该方法利用了操作系统中“资源有序分配”和“资源请求限制”的思想，虽然牺牲了一部分并发性，但确保了系统的安全性和稳定性。

● 实验步骤：

1. 哲学家吃饭问题（死锁情况，不解决）

1.1 代码

每个哲学家进程

```
void Person1(void *arg)
{
    for(int e = 0; e < 20; e++){
        chopsticks_1.P();
```



```
wait();

chopsticks_5.P();

printf("1  ");

wait();

chopsticks_5.V();

chopsticks_1.V();

}
```

1.2 结果展示



有五位哲学家（1，2，3，4，5），程序输出 1 表示 1 号哲学家吃饭，只输出一
次 1 后产生死锁问题，程序崩溃。

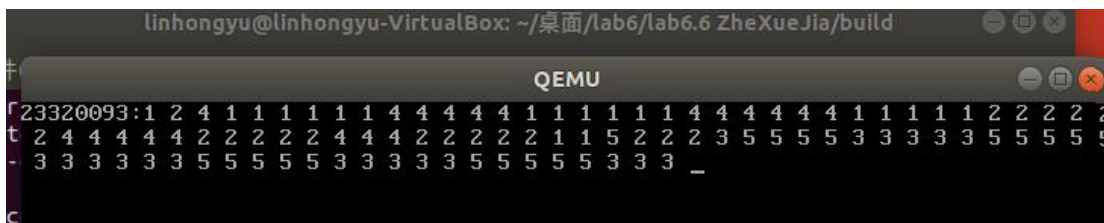
每个哲学家都拿了一个筷子，产生死锁问题。

2. 解决哲学家吃饭死锁问题

2.1 代码

```
void Person1(void *arg)
{
    for(int e = 0; e < 20; e++){
        chopsticks_5.P(); //左边筷子
        wait();
        chopsticks_1.P(); //右边筷子
        printf("1 ");
        wait();
        chopsticks_1.V();
        chopsticks_5.V();
    }
}
```

2.2 结果展示



不会出现死锁 程序一直运行

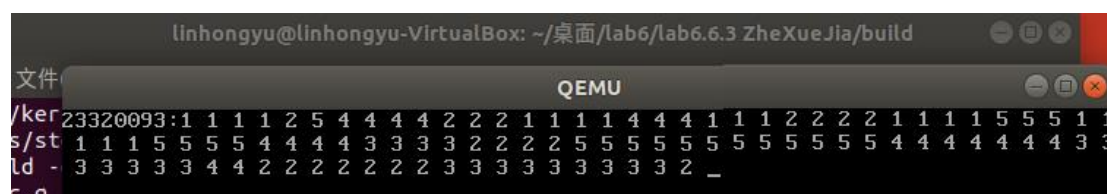
3. 死锁应对策略一

3.1 代码

进入程序之后先判断每次在餐桌上的人数，餐桌已有四个人则不上桌吃饭，继续等待少于四个人再吃饭。

```
void a1(void *arg)
{
    for(int e = 0; e < 20; e++){
        wait();
        if(people_nums<=4)
        {
            people_nums++;
            avil_5.P();
            wait();
            avil_1.P();
            printf("1 ");
            wait();
            avil_5.V();
            avil_1.V();
            people_nums--;
        }
        else
            e--;
    }
}
```

3.2 结果展示



Section 5 互斥与锁方法总结

在操作系统中，互斥是为了防止多个线程或进程同时访问共享资源，从而引发数据不一致或冲突的问题。

常用的互斥机制包括：

信号量：一种整型变量，用于控制多个线程对共享资源的访问，通过 P（等待）和 V（释放）操作实现资源控制。本实验中使用的是二进制信号量，确保筷子（资源）一次只能被一位哲学家使用。

互斥锁：类似于二进制信号量，通常用于保护临界区代码，确保同一时间只

有一个线程进入关键区。

自旋锁：线程在等待锁时会不停地循环检查锁状态，适用于等待时间短的场景。

条件变量、读写锁等：用于更复杂的同步需求，如读写资源分离。

Section 5 实验总结

本实验通过模拟消费者生产者、哲学家就餐问题，深入理解了操作系统中进程同步与互斥机制的实现与应用。通过引入信号量，解决了哲学家因竞争资源导致的死锁问题。实验中先实现了死锁情形，再通过调整筷子获取顺序以及应用抽屉原理进行优化，验证了不同同步策略对系统安全性的影响。本实验不仅加深了对信号量操作的理解，还提升了实际编程中解决并发问题的能力。